



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Packet Headers

CMSC 131 Lab 1

Prepared by: Rene Andre Bedonia Jocsing

Published: September 18, 2026

Deadline: December 09, 2026

This laboratory activity is about reading and writing a binary format. You'll build `renpkt`, a tool that takes the twenty bytes at the front of an IPv4 packet, pulls every field out of them, puts them back, and computes the checksum that proves the result is intact.

The activity covers exactly one format: the 20-byte IPv4 base header. Those twenty bytes hold the thirteen fields in the table below and nothing else. IPv4 options are outside the activity, along with the longer headers that carry them. No sample has one. The encoder never writes one.

Background

Every IPv4 packet that crosses the internet carries a header describing where it came from, where it's going, and how to handle it along the way. This header is a fixed sequence of bytes with a layout published in 1981 as Request for Comments 791, or RFC 791, the document that defines the Internet Protocol. Every IPv4 router agrees on it, because a router that read the bytes differently would send the packet somewhere else.

These fields don't respect byte boundaries. A version number occupies four bits. A fragment offset occupies thirteen, straddling the boundary between two bytes. You can't read these with a `mov`. There's no instruction that loads thirteen bits. You have to shift and mask, which is Block 7 material applied to something that wasn't invented for a classroom.

The multi-byte numbers use a different byte order from the one your registers use. Take the number 60, which is `0x003C` as a 16-bit value. Your processor is little-endian: it stores the low-order byte first, so a `mov` of that value into memory writes `3C 00`. The network is big-endian: the high-order byte comes first, so the same number travels as `00 3C`. Neither order is wrong. They're two conventions for the same two bytes. The header uses the network one, so every 16-bit field arrives with its bytes in the order your registers don't expect. The conversion is your job.

Learning Objectives

By the end of this laboratory activity, students should be able to:

- Extract bit fields of arbitrary width and position from a packed binary structure
- Handle a field that straddles a byte boundary without corrupting its neighbours
- Convert between little-endian and big-endian byte order by hand
- Implement the one's complement checksum used throughout the internet protocols
- Build a multi-file NASM project driven by a `Makefile`

- Interface assembly routines with a C driver that handles file input and output

Task

Build a tool (`renpkt`) that:

- Reads a binary file containing a 20-byte IPv4 header
- Decodes all thirteen header fields and prints them in a readable form
- Verifies the IPv4 header checksum and reports whether it's valid
- Builds a new header from field values supplied on the command line
- Computes and inserts the correct checksum into a header it builds

Required Features

The Header Layout

Twenty bytes, with bit positions counted from the highest-order bit of each byte.

byte	0	1	2	3
0	Ver IHL DSCP ECN	Total Length		
4		Identification	Flg	Frag Offset
8		TTL	Protocol	Header Checksum
12		Source Address		
16		Destination Address		

Field by field:

Field	Location	Width	Notes
Version	byte 0, bits 7-4	4	Always 4 for IPv4
IHL	byte 0, bits 3-0	4	Internet Header Length, in 32-bit words, so multiply by 4 for bytes. Always 5 here
DSCP	byte 1, bits 7-2	6	Differentiated Services Code Point, the traffic class
ECN	byte 1, bits 1-0	2	Explicit Congestion Notification
Total Length	bytes 2-3	16	Big-endian. Counts the header and the payload, so 20 is the smallest legal value and 65535 the largest
Identification	bytes 4-5	16	Big-endian

Field	Location	Width	Notes
Flags	byte 6, bits 7-5	3	Bit 7 is reserved and must be zero. Bit 6 is Don't Fragment (DF). Bit 5 is More Fragments (MF)
Fragment Offset	bytes 6-7, low 13 bits	13	Big-endian, straddles a byte boundary
TTL	byte 8	8	Time To Live, the hop limit
Protocol	byte 9	8	6 is TCP, 17 is UDP
Header Checksum	bytes 10-11	16	Big-endian, computed as below
Source Address	bytes 12-15	32	Four octets
Destination Address	bytes 16-19	32	Four octets

The Supported Field Set

The thirteen fields in that table are the whole supported set: version, IHL, DSCP, ECN, total length, identification, flags, fragment offset, TTL, protocol, header checksum, source address, and destination address. The supported header is always twenty bytes. The IHL is always 5.

Four things sit outside the activity:

- IPv4 options. A header carries them only when the IHL is above 5, and every call in this activity works on the 20-byte base header.
- Any header longer than twenty bytes. `renpkt` reads twenty bytes and writes twenty bytes.
- The version field's value. The encoder always writes 4.
- The payload. `renpkt` never looks past byte 19.

Two fields have a legal range narrower than their width. The total length counts the header itself, so a value below 20 describes a packet shorter than its own header. The driver accepts 20 through 65535. The top flag bit is reserved by RFC 791 and must be zero, so the flags field holds one of four values: 0, MF alone (1), DF alone (2), or both (3). The driver rejects 4 through 7. Your decode path still reads whatever three bits the file holds. Enforcing the standard on the way out is the driver's job, done before your code runs.

Decoding

```
1 ./renpkt --decode tests/sample01.bin
```

```
1 Version:          4
2 IHL:              5 (20 bytes)
3 DSCP:             0
4 ECN:              0
5 Total Length:     60
6 Identification:   7238
7 Flags:            2 (DF)
8 Fragment Offset:  0
9 TTL:              64
10 Protocol:         6 (TCP)
11 Header Checksum:  0x9CBC
12 Source:           192.168.0.104
13 Destination:     192.168.0.1
14 Checksum:         VALID
```

That output corresponds to these twenty bytes, which is `tests/sample01.bin`:

```
1 45 00 00 3c 1c 46 40 00 40 06 9c bc c0 a8 00 68 c0 a8 00 01
```

Trace the first byte yourself before writing any code. `0x45` is `0100 0101`. The high nibble is `0100`, decimal 4, the version. The low nibble is `0101`, decimal 5, the IHL. Five 32-bit words is twenty bytes, which is the length of the header you're holding.

Encoding

```
1 ./renpkt --encode --ttl 64 --proto 6 --len 60 --id 7238 \
2     --src 192.168.0.104 --dst 192.168.0.1 --df \
3     -o out.bin
```

The tool writes twenty bytes with every field in its correct position and byte order, computes the checksum, and inserts it. Decoding `out.bin` must reproduce the field values you supplied, and must report the checksum as valid. Those particular values rebuild `sample01.bin` byte for byte, which is a check you can run right now with `cmp`.

`--len` defaults to 20 when you leave it out, the length of a header with nothing behind it. Every other option defaults to zero.

The IPv4 Header Checksum

The IPv4 header checksum is the one's complement of the one's complement sum of every 16-bit word in the header.

Concretely:

1. Treat the header as ten 16-bit big-endian words.
2. Treat the checksum field itself as zero while computing.
3. Add all ten into a 32-bit accumulator.
4. While the accumulator exceeds 16 bits, fold: add the high half to the low half. This is the "end-around carry" and it's what makes the sum a one's complement one.
5. Take the bitwise NOT of the low 16 bits. That's the checksum.

Worked against `sample01.bin`, with the checksum field zeroed:

```
1 4500 + 003c = 453C
2 453C + 1C46 = 6182
3 6182 + 4000 = A182
4 A182 + 4006 = E188
5 E188 + 0000 = E188      (checksum field, treated as zero)
6 E188 + C0A8 = 1A230     -> fold: A230 + 1 = A231
7 A231 + 0068 = A299
8 A299 + C0A8 = 16341     -> fold: 6341 + 1 = 6342
9 6342 + 0001 = 6343
10
11 NOT 6343 = 9CBC
```

Which matches bytes 10 and 11 of the sample.

The example folds each time the sum overflows. Folding at the end gives the same answer as folding inside the loop, as long as you keep folding until the accumulator fits in 16 bits. Most headers need one fold. A sum of `0x8FFFF` needs two because the first fold leaves `0x10007` and the carry has to come around again. `contract_test` ships that vector, and a routine that folds once returns `0xFFFF8` where it owes `0xFFFF7`.

Verification Has a Shortcut

Running the same computation over a header that *already contains* its checksum gives zero. That's a property of one's complement arithmetic. It's how routers check headers without recomputing and comparing.

So your `--decode` path doesn't need to compute the expected checksum and compare it. It sums the header as it stands, including the checksum field, and reports valid when the result is `0x0000`. Implement it that way.

The Checksum After a Header Change

The sum covers the complete header every time. A router that changes a field, the TTL being the usual one, has to leave the checksum describing the header as it now stands. It recomputes the sum over the whole header, or it applies the equivalent correction for the one field that changed. Both roads reach the same stored value. The definition is the whole header. Nothing may patch the checksum from a partial sum that no longer matches the bytes on the wire.

Before Week 1: The C Boundary

This activity opens on September 18. The lecture that covers calling conventions, L9, lands on September 29, eight days after the first progress report window opens. You can't wait for it. This section is the bridge: everything you need to call assembly from C and C from assembly, taught here so that nothing in the Week 1 report depends on a lecture you haven't had. L9 goes deeper. Read this first anyway.

Why a Convention at All

`driver.c` is compiled by `gcc`. Your routines are assembled by `NASM`. Neither tool reads the other's source. For a call across that line to work, both sides have to agree, in advance, on where the arguments are, where the answer goes, and which registers the callee may destroy. That agreement is a **calling convention**. The one this course uses on 32-bit x86 is called **cdecl**, and C compilers on this platform use it by default.

How C Calls Your Routine

Take the decode path. `driver.c` reads twenty bytes into a buffer, clears a struct, and makes this call:

```
1 decode_header(hdr, &f);
```

Here's what `gcc` emits for that line, in the assembly you already know:

```
1      push    dword [address of f]      ; the second argument, pushed first
2      push    dword [address of hdr]    ; the first argument, pushed last
3      call    _decode_header
4      add     esp, 8                    ; the caller removes both arguments
```

Follow `esp` through it. Suppose `esp` is `0x1000` before the first push. Each push subtracts four and writes:

```
1      push &f      esp = 0x0FFC  [0x0FFC] = &f
2      push hdr     esp = 0x0FF8  [0x0FF8] = hdr
3      call         esp = 0x0FF4  [0x0FF4] = the return address
```

Now you're inside `_decode_header`, and the first thing the stub does is `enter 0,0`, which is `push ebp` followed by `mov ebp, esp`:

```
1  push ebp      esp = 0x0FF0  [0x0FF0] = the caller's ebp
2  mov ebp, esp  ebp = 0x0FF0
```

Read the stack from `ebp` upward. `[ebp]` is the saved `ebp`. `[ebp+4]` is the return address. `[ebp+8]` is `hdr`, the first argument. `[ebp+12]` is `&f`, the second. That's the whole rule. After `enter 0,0`, the first argument is at `[ebp+8]` and each later one is four bytes higher. Right-to-left pushing is what puts the first argument nearest the frame.

Return values go in `eax`. `ip_checksum` returns an unsigned `short`, so the caller reads `ax` and ignores the top half. The two `void` routines return nothing. Their stubs' `mov eax, 0` is harmless.

`leave` undoes `enter`: it's `mov esp, ebp` then `pop ebp`, which throws away anything you pushed inside the routine and restores the caller's frame. `ret` pops the return address. Back in `driver.c`, `add esp, 8` discards the two arguments. Under `cdecl` the *caller* cleans up. Your routine returns with the arguments still on the stack and `esp` where `call` left it.

Who Saves What

Eight registers. Seven of them sort into two groups, and `esp` is its own case, covered below. `eax`, `ecx`, and `edx` are **caller-saved**: the caller assumes a call may destroy them, so it keeps nothing important there across a `call`. `ebx`, `esi`, `edi`, and `ebp` are **callee-saved**: the caller assumes they come back unchanged, so if your routine uses one, your routine has to put it back.

You met this promise in Block 7 from the other side. `pusha` and `popa` were how your `_asm_main` handed every register back to `driver.c`. The block called it a promise to whoever called you. That was the conservative way to keep it: save all eight, restore all eight, think about none of them. `cdecl` states the promise precisely. Only four registers are owed, and of those, only the ones you change need saving. The usual form pushes the ones you use, `ebx`, `esi`, `edi`, or any of them, right after `enter`, and pops them in reverse order right before `leave`. `enter` and `leave` handle `ebp` between them. `pusha/popa` still works, and the stubs ship with it, but it saves `eax` too, so a routine that returns a value in `eax` must set it *after* the `popa`. Block 7 explained that ordering. It applies here to `ip_checksum`.

The fifth obligation is `esp`. It's balanced rather than saved: every push inside your routine has a matching pop, or `leave` cleans up after you, so the `ret` runs with `esp` pointing at the return address. A routine that ends with `ret 8` pops its own arguments. That's a different convention, `stdcall`, and under it the caller's `add esp, 8` removes eight bytes that were never its arguments.

The provided `contract_test` checks all five. It calls each routine with known values in `ebx`, `esi`, and `edi`, records `ebp` and the expected `esp`, and prints the register of each obligation that came back broken.

The Struct

`driver.c` hands you a pointer to a `struct ipv4_fields`. A C struct is a sequence of members laid out in memory in declaration order. This one has thirteen: eleven unsigned `int`, four bytes each, then two arrays of four unsigned `char`. No padding. Every `int` member is four bytes, so the offsets are multiples of four:

```
1  +0 version          +4  ihl
2  +8 dscp             +12 ecn
3  +16 total_length    +20 identification
4  +24 flags           +28 fragment_offset
5  +32 ttl             +36 protocol
6  +40 checksum
7  +44 src[0] src[1] src[2] src[3]
8  +48 dst[0] dst[1] dst[2] dst[3]
```

So with the struct's address in `edi`, the total length lives at `[edi+16]` and a 32-bit store fills it: `mov [edi+16], eax`. The octets are single bytes: `mov [edi+44], al` stores the first octet of the source address. `driver.c` has the same table in a comment. When the two disagree, the struct in `driver.c` wins.

Two Spellings of One Name

On Windows, the C compiler puts `_` in front of every external symbol, so the function C calls `decode_header` is the symbol `_decode_header`. On Linux it doesn't, so the symbol is `decode_header`. The stubs handle this for you with a block at the top of each file:

```
1  #ifndef ELF_TYPE
2  #define _decode_header decode_header
3  #endif
```

The Makefile passes `-d ELF_TYPE` on Linux and nothing on Windows. You write `_decode_header` everywhere, and on Linux the `#define` respells it. Leave that block alone. `asm_io.inc` did the same thing for `_asm_main` in the bootcamp, which is why your bootcamp programs built on both platforms without you noticing.

Assembly Calling Assembly

`encode_header` has to call `ip_checksum`. The call goes the same way C's did, with you as the caller: push the arguments right to left, `call`, clean up.

```
1      push     dword 20                ; len, the second argument
2      push     edi                    ; hdr, the first argument
3      call     _ip_checksum
4      add      esp, 8                 ; you pushed eight bytes, you remove them
5      ; ax now holds the checksum
```

`ip_checksum` may destroy `eax`, `ecx`, and `edx`. It must not touch `ebx`, `esi`, `edi`, or `ebp`, so a pointer you're keeping in `edi` survives the call. That's the same promise seen from the other side of the line. It's also why `encode.asm` declares `_ip_checksum` as `extern`. The two routines live in different files, and the linker joins them.

What Week 1 Asks

The first progress report asks you to explain this section. State the five things a `cdecl` routine owes its caller. Trace one call from `driver.c` to your stub and back, with `esp` at each step. Show where the struct offsets in your comments came from. Nothing in the Week 1 report requires a working routine, but a working `decode_header` for byte 0 is the natural prototype. The Progress Reports section near the end lists the rest of what Week 1 wants, and what the two weeks after it want.

Technical Requirements

Getting the Starter

Fork [cmssc-131-lab1-starter](https://github.com/cmssc-131-lab1-starter) on GitHub and clone your fork. A fork is a snapshot. A fix pushed to the starter mid-week never reaches it, so the fixtures can't change under you.

```
1  git clone https://github.com/<your-account>/cmssc-131-lab1-starter.git
2  cd cmssc-131-lab1-starter
```

```

3  make
4  make check

```

The assembly files ship as stubs that assemble and link as-is, so the build works before any code is written and `make check` fails on every test: 7 of 7 checks differ. That red run is the correct starting state.

The provided files in the tree below are fixtures. The grader compares your fork against the starter, so an edited `driver.c`, `Makefile`, `run_tests.sh`, `contract_test.c`, `contract_regs.asm`, or provided `tests/` file shows up as a diff in the open. One member forks the repository under their account, keeps it public, and adds the rest of the group as collaborators, since that fork is the repository you submit.

Project Structure

```

1  renpkt/
2    Makefile
3    driver.c           provided: argument parsing, file I/O
4    cdecl.h           provided: the calling-convention macros
5    decode.asm        yours
6    encode.asm        yours
7    checksum.asm      yours
8    contract_test.c   provided: the second gate, in C
9    contract_regs.asm provided: register checks for contract_test
10   tests/
11     manifest.txt     provided: every header below, marked valid or invalid
12     sample01.bin     provided
13     sample02.bin     provided
14     sample03.bin     provided
15     sample04.bin     provided: non-zero fragment offset, MF set
16     sample05.bin     provided: the boundary header, described below
17     bad01.bin        provided: deliberately corrupted
18     expected/        provided: expected stdout per test
19     run_tests.sh     provided
20     README.md        yours: design notes, subsystem ownership, quirks and issues

```

The starter's `README.md` ends with three sections that ship with prompts and empty tables or lists under them: Design Notes, Subsystem Ownership, and Quirks and Issues. Each one says which progress report it's due for. Design Notes and Subsystem Ownership are Week 1 deliverables under the syllabus. Quirks and Issues is Week 3's. The rest of the `README` is the repository's own notes on the gate and the fixtures. Leave those alone.

The C Boundary

The provided `driver.c` handles everything that isn't bit manipulation. It parses the command line, opens files, reads the twenty bytes into a buffer, and calls into your assembly:

```

1  /* Fills the caller's field struct from a 20-byte header. */
2  void PRE_CDECL decode_header(unsigned char *hdr, struct ipv4_fields *out) POST_CDECL;
3
4  /* Writes 20 bytes into hdr from the field struct, checksum included. */
5  void PRE_CDECL encode_header(struct ipv4_fields *in, unsigned char *hdr) POST_CDECL;
6
7  /* Returns the IPv4 header checksum over len bytes. */
8  unsigned short PRE_CDECL ip_checksum(unsigned char *hdr, int len) POST_CDECL;

```

You implement those three. Don't modify `driver.c`, since your defense will use the provided copy.

`ip_checksum` is defined for a nonnegative, even byte count because the sum walks the header two bytes at a time. Every call in this activity passes 20, the length of the base header. You don't have to invent a meaning for a negative or odd length.

Arguments arrive on the stack under the `cdecl` convention. The first argument sits at `[ebp+8]`, the second at `[ebp+12]`, after a standard `enter 0,0` prologue. Return values go in `eax`. The section above walks through why.

Preserve `ebx`, `esi`, `edi`, and `ebp` across your routines, and return with `esp` where the call left it. C assumes all five. A routine that breaks one produces failures far from their cause.

Constraints

- No `bswap` and no `xchg`. Byte reversal must be written with shifts and masks. The instruction exists. Doing it by hand is the exercise.
- No lookup tables for the checksum. Compute it.
- Whole-number arithmetic only. No floating point, no FPU.
- Every field must be extracted from the buffer by your assembly, with shifts and masks. There's no other route. `driver.c` is a fixture, and the `Makefile` links exactly three objects of yours.
- Callee-saved registers must be preserved. `contract_test` checks it on every `make check`. The wrapper it links in, `contract_regs.asm`, calls each of your routines with sentinel values in the registers, and the defense runs it again on the commit you defend.

Implementation Notes

Reading a 16-bit Big-Endian Field

Bytes 2 and 3 hold the total length, highest-order byte first. Loading them as a 16-bit value gives you the bytes in the other order, so recombine them explicitly:

```

1 ; esi points at the header, we want bytes 2-3 as a number
2     movzx  eax, byte [esi + 2]      ; high byte
3     shl    eax, 8
4     movzx  ebx, byte [esi + 3]      ; low byte
5     or     eax, ebx                ; eax = the 16-bit value

```

`movzx` loads a byte and zero-extends it to fill the register, which saves you clearing the upper bits by hand.

Reading it byte by byte sidesteps the endianness question. You never load a multi-byte network field directly.

The Field That Straddles

Bytes 6 and 7 hold three flag bits followed by a thirteen-bit fragment offset:

```

1      byte 6          byte 7
2  +---+---+---+---+---+
3  | R | DF| MF| off | offset |
4  +---+---+---+---+---+
5      bit 7,6,5    4..0    7..0

```

The offset's top five bits live in byte 6 and its bottom eight in byte 7. Neither byte holds the whole field, so a mask applied to one byte can't produce it. Combine the two bytes into one 16-bit word first. Then a single mask, `0x1FFF`, takes the offset out of that word in one step.

```

1      movzx    eax, byte [esi + 6]
2      shl     eax, 8
3      movzx    ebx, byte [esi + 7]
4      or      eax, ebx                ; eax = the whole 16-bit word
5
6      mov     ebx, eax
7      shr     ebx, 13                ; ebx = flags, the top 3 bits
8      and     ebx, 0x07
9
10     and     eax, 0x1FFF            ; eax = fragment offset, low 13 bits

```

Building the same word for `--encode` is the reverse: shift the flags up by 13, mask the offset to 13 bits, or them together, then split into two bytes.

The three flag bits, read as a number, are what `driver.c` stores in the `flags` member and prints. `sample01.bin` has DF set and nothing else, so the field is 010, which prints as 2 (DF). The reserved bit is the top one. The encoder never sets it and no valid sample has it set. Your decoder still reads all three bits as the file holds them.

Why 0x1FFF

Thirteen bits set is 0001 1111 1111 1111, or 0x1FFF. A mask one bit too wide reads the MF flag into the offset. A zero fragment offset looks correct either way because the extra bits are zero, so test with a non-zero offset. `sample04.bin` has one, and `sample05.bin` has the largest one the field can hold.

The Checksum Loop

```

1  ; esi = header, ecx = length in bytes, returns checksum in ax
2  ; the loop changes ebx and esi, so save both before it and restore them after
3      xor     eax, eax                ; accumulator
4  sum_loop:
5      cmp     ecx, 0
6      jle     fold
7      movzx   ebx, byte [esi]
8      shl     ebx, 8
9      movzx   edx, byte [esi + 1]
10     or      ebx, edx
11     add     eax, ebx                ; 32-bit accumulator, carries are kept
12     add     esi, 2
13     sub     ecx, 2
14     jmp     sum_loop
15  fold:
16     ; while (eax >> 16) != 0: eax = (eax & 0xFFFF) + (eax >> 16)
17     ...
18     not     eax
19     and     eax, 0xFFFF

```

Accumulate in 32 bits and fold at the end, as many times as it takes for the accumulator to fit in 16 bits. The worked example folds inside the loop instead. Same answer, a few more instructions. What you can't do is mask the accumulator to 16 bits without folding. That drops every carry silently. The checksum then comes out wrong on any header whose sum overflows.

Common Pitfalls

- Using `bswap`, which is banned here and will be checked at defense

- Masking the fragment offset with 0xFFF or 0x3FFF rather than 0x1FFF
- Forgetting that IHL counts 32-bit words, so a value of 5 means 20 bytes
- Including the checksum field's stored value when computing a checksum for --encode, when it must be treated as zero
- Folding the carry only once when a large sum needs folding twice
- Loading a 16-bit field with a single `mov` and getting the bytes reversed
- Clobbering `ebx`, `esi`, `edi`, or `ebp`, or returning with `esp` moved, and breaking the C caller
- Setting `eax` before the `popa` in a routine that returns a value, so the `popa` overwrites the answer
- Comparing your computed checksum against the stored one on the decode path, when summing the whole header and testing for zero is simpler and is what the specification asks for

Testing Strategy

Correctness

Six headers are provided. `tests/manifest.txt` lists each one as valid or invalid. `sample01.bin` through `sample05.bin` are valid. `sample04.bin` carries a non-zero fragment offset and the More Fragments flag, so a wrong mask fails it. `sample05.bin` is the boundary header: DSCP 63, ECN 3, total length 65535, identification 65535, both DF and MF set, fragment offset 8191, TTL 255, protocol 255, and a source address of 255.255.255.255. Every field the standard lets you fill is full, so a mask one bit too narrow or one bit too wide fails it. Version and IHL stay at 4 and 5. The reserved flag bit stays zero. Any other value there would make the header illegal, and the point of a boundary fixture is a legal header at the edge. `bad01.bin` has one byte corrupted and must be reported invalid.

```
1 make check
```

`check` builds the tool and the contract test, then runs `run_tests.sh`, which reports each case and exits nonzero when any of them differ. The gate has two passes.

The decode pass runs every header the manifest lists through `--decode` and compares the output against its file in `tests/expected/`, byte for byte. The comparison uses `--strip-trailing-cr`, for the reason Block 1 explained.

The contract pass runs `contract_test`, which decodes and re-encodes every valid header in the manifest and compares the bytes, checks the checksum vector that needs two carry folds, and checks that all three routines keep `ebx`, `esi`, `edi`, and `ebp` and return with `esp` in place. That pass catches what the output comparison can't see.

(`make test` still works, as an alias for `check`.)

Round Trip

The strongest test you can run, the one your defense will use. `sample04.bin` is the interesting case, because it carries a non-zero fragment offset, the MF flag, and a non-zero DSCP with a non-zero ECN:

```
1 ./renpkt --decode tests/sample04.bin > before.txt
2 ./renpkt --encode --dscp 46 --ecn 1 --len 1500 --id 48879 \
3     --mf --frag 4660 --ttl 128 --proto 17 \
4     --src 10.0.0.1 --dst 10.0.0.2 -o rebuilt.bin
5 cmp rebuilt.bin tests/sample04.bin
```

Those are the field values `--decode` prints for `sample04.bin`, fed back in. `cmp` compares the two files byte for byte. It prints nothing when they match. A rebuilt file that differs means the encoder lost

or moved a field. The command `cmp -l rebuilt.bin tests/sample04.bin` identifies the byte where it happened.

Decode, re-encode from what you decoded, then compare the bytes. Repeat the decode as a second opinion if you like, but the byte comparison is the one that catches an encoder bug that decoding alone will never reveal. `contract_test` makes the same comparison for every valid header in the manifest, so `make check` fails on an encoder bug even when every decode looks right.

Adding Headers

Put a new header in `tests/`, write the output `--decode` must print for it in `tests/expected/`, and add one line to `tests/manifest.txt` naming it `valid` or `invalid`. A valid header you add joins the round trip automatically. The gate fails on a `.bin` that isn't in the manifest, and on a listed header without an expected file. Its message says which file. The shipped corpus covers most of the cases below already. `sample05.bin` fills every field the standard lets you fill, which is the worst case for a mask of the wrong width, and `contract_test` ships the vector whose checksum needs the carry folded twice. For the rest, build headers of your own and confirm they survive a round trip:

- Fragment offset of 0, of 1, and of 8191, its maximum
- Each legal flag combination: neither flag, DF alone, MF alone, and both
- Total length of 20, the minimum, and 65535, the maximum
- TTL of 1 and of 255
- DSCP of 63 and ECN of 3, the two fields that share byte 1, both at their maximum
- A checksum whose computation needs the carry folded twice

Deliverables

Your group's GitHub repository, with a clean commit history showing individual contributions, must contain:

1. `decode.asm`, `encode.asm`, and `checksum.asm`, documented
2. The provided `Makefile`, `driver.c`, `run_tests.sh`, `contract_test.c`, `contract_regs.asm`, and `tests/`, unmodified, plus any headers you added and their lines in `tests/manifest.txt`
3. `README.md`, with its Design Notes, Subsystem Ownership, and Quirks and Issues sections filled in by the progress report each one is due for

Subsystem Ownership

The activity splits into three subsystems, one per member. In a group of four, two members share one. Record the split in the Subsystem Ownership section of `README.md` before the Week 1 progress report.

Subsystem	Covers
Decode path	Reading the buffer, extracting all thirteen fields, filling the struct
Encode path	Building the 20 bytes from field values, byte ordering, insertion
Checksum and tests	The checksum routine, the test harness, added headers and their manifest lines

You'll be asked at defense about a subsystem you didn't write.

To submit, one designated member emails the group submission, sending the link to your group's public GitHub repository. CC the rest of your group. The commit you defend is the one that's graded, so push before your defense slot. Your provisional grade comes from that commit. The syllabus then gives you one week to commit any refactors your instructor requires at the defense. Those commits can

raise the grade. Nothing else you push after the defense changes it. Adhere to the following subject line, joining every group member's name with & in the same LastName, Initials format: [CMSC 131 Lab] Lab 1 Group: LastName1, Initials1 & LastName2, Initials2 & LastName3, Initials3. A quadro adds a fourth name the same way.

Then, each member submits, individually, through email:

1. A short `reflection.txt` covering what you built on your subsystem, what fought back, and what you learned.
2. A short `peer.txt` with your honest assessment of how your other groupmates worked during the activity.

Adhere to the following subject line: [CMSC 131 Lab] Lab 1: Surname, Initials.

For example: [CMSC 131 Lab] Lab 1 Group: Sanchez, SM & Reyes, JD & Cruz, AB for the group submission and [CMSC 131 Lab] Lab 1: Sanchez, SM for the individual submission.

Progress Reports

The syllabus says what every progress report expects in general terms: design notes, the subsystem split, and a working prototype in Week 1, a functional prototype with the core features in Week 2, and the complete implementation with test results and documented quirks in Week 3. This section says what those mean for this activity. Each report is a 15-minute slot. Come with the repository open and the tool built, and expect every member to speak.

Week 1

The C boundary, and a plan. L9 hasn't happened yet, so everything Week 1 asks about comes from "Before Week 1: The C Boundary" above or from Blocks 1 through 7. Bring:

- The Subsystem Ownership section of `README.md`, filled in, and a commit history that already shows who owns what
- The Design Notes section of `README.md`: the header layout in your own words, how the three routines split the work, and a timeline naming the subsystem each week finishes
- A prototype that builds and runs on a provided sample. A `decode_header` that fills version and IHL from byte 0 and nothing else is the natural one
- The five things a `cdecl` routine owes its caller, and a trace of one call from `driver.c` into your stub with `esp` at each step
- Where the struct offsets in your comments came from, and how you extract a field narrower than one byte

Week 2

The decode path, end to end. Bring:

- `--decode` on every valid sample, `sample01.bin` through `sample05.bin`, printing the right fields
- `bad01.bin` reported invalid
- The fragment offset extracted with a thirteen-bit mask, and the reason it's thirteen
- A checksum routine that folds the carry, and an encoder that zeroes the checksum field before it computes
- A commit history spread across the week. Each member opens code they wrote and explains a routine inside it

Week 3

Everything green, and the paper trail. Bring:

- `make check` passing all seven checks, the six headers and the contract pass
- A round trip on a provided sample, with `cmp` showing no difference

- Headers of your own beyond the provided ones, listed in `tests/manifest.txt`, including the maximum fragment offset and a checksum that needs the carry folded twice
- The Quirks and Issues section of `README.md`, filled in, with at least one entry you can explain
- The provided files unmodified, and three weeks of commits distributed across every member

Week 4

The defense. It takes a 25-minute slot. Your instructor runs the tool against its reference and questions each member on a subsystem they didn't write. The commit you defend is the one that's graded.

Academic Honesty

The usage of Large Language Models (e.g. ChatGPT, Claude, DeepSeek, etc.) to generate wholesale vibe-coded or vibe-written work is considered cheating. Cheating is against the university's code of ethics. It's subject to failure in the course, harsh disciplinary action, or expulsion. The syllabus's Academic Integrity and Honesty section is the policy authority. It applies in full.

Important Dates

This activity has no deadline of its own. It has a prescribed timeline, the four windows below, and the syllabus attaches extra credit to keeping it. The defense belongs in the Week 4 window. The one hard cutoff is course-wide: after the end of classes on December 9, no laboratory deliverable is accepted for any activity.

Progress reports and laboratory defense may be booked only during the hours defined in the syllabus, with your own laboratory instructor, on that instructor's booking page. The syllabus lists one page per instructor. Book ahead.

Activity	Window	Sec 3 (W)	Sec 1 & 2 (T/F)
Week 1 Progress Report	Sept 21-27	Sept 23	Sept 22, 25
Week 2 Progress Report	Sept 28-Oct 4	Sept 30	Sept 29, Oct 2
Week 3 Progress Report	Oct 5-11	Oct 7	Oct 6, 9
Defense Window	Oct 12-18	Oct 14	Oct 13, 16

Grading Rubric (100 Pts)

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
System Architecture (25%)	Clean separation between <code>decode_header</code> , <code>encode_header</code> , and <code>ip_checksum</code> . No duplicated bit-extraction logic. Register usage planned rather than improvised.	Mostly modular. Some coupling between routines. Register choices workable but ad hoc.	Sprawling code. One monolithic routine. The same shift-and-mask pattern copied into all three.	No modularity. Logic interleaved arbitrarily. Violates the <code>cdecl</code> boundary.
Robustness (20%)	Handles every edge case including maximum fragment offset, double-folded carries, and corrupted headers. Round trip is exact.	Core cases correct. Minor issues at field boundaries or with unusual flag combinations.	Valid samples decode, but encoding loses fields or the checksum is wrong for some inputs.	Fails the provided samples, or reports a corrupted header as valid. The corrupted header is a case to detect, not a case to survive.
Code Engineering (10%)	Callee-saved registers preserved throughout, and the contract pass proves it. No magic numbers without a named reason. Comments explain why a mask is that width.	Registers preserved. A few unexplained constants.	Inconsistent register discipline. Masks and shifts uncommented.	Clobbers registers the C caller needs. Unreadable throughout.
Collaboration (20%)	Atomic semantic commits distributed across the month. Clear per-member ownership. Evidence of real code review between members.	Consistent version control. Adequate messages. Work visibly shared.	Inconsistent Git usage. Large dumps instead of increments. Vague messages.	Minimal history. No evidence of the whole group contributing.
Technical Defense (25%)	Every member explains the bit-level mechanics of all three routines. Each can trace a field they didn't write. Handles what-if questions confidently.	Clear explanation. All members participate meaningfully. Sound reasoning.	Uneven participation. Struggles to trace the straddling field by hand.	Can't explain the extraction or the checksum. Unable to defend design decisions.